



PROPUESTA DE AUTOMATIZACIÓN DEL CICLO DE DESARROLLO

Guía Generativa de Desarrollo

Convertir la fase de desarrollo de una task — desde que el equipo la recibe hasta dejar el PR abierto en GitHub — en un proceso ejecutable por IA: verificable, auditable y repetible.

Viva Aerobus

Proyecto Generic API

Plan de trabajo — 16 semanas

DOCUMENTO CONFIDENCIAL. ELABORADO POR TECH AND SOLVE PARA VIVA AEROBUS.

CONTENIDO

- 1. Resumen ejecutivo
- 2. El problema que resolvemos
- 3. Qué queremos lograr
- 4. El flujo de trabajo completo, paso a paso
- 5. Las fases, y por qué en ese orden
- 6. Detalle de cada fase y estimación
- 7. Cronograma y esfuerzo
- 8. Cómo mediremos que funcionó
- 9. Capacidad designada para soporte y evolución
- 10. De qué depende
- 11. Cambios que requiere la dinámica de trabajo
- 12. Riesgos y mitigación
- 13. Próximos pasos

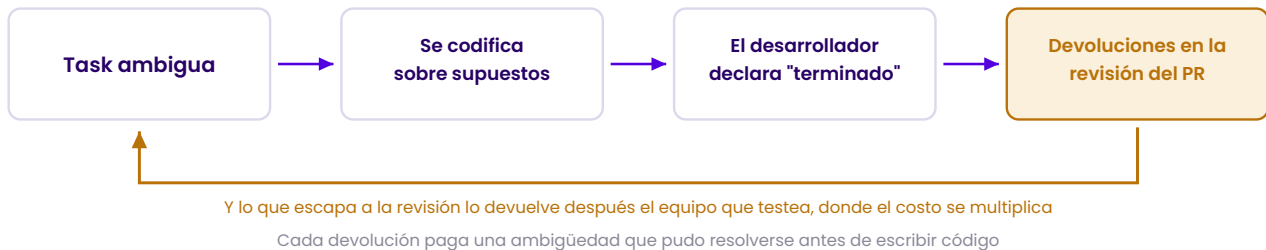
1. Resumen ejecutivo

Tech and Solve pone a consideración de Viva Aerobus esta propuesta para el proyecto **Generic API**. La intención es concreta: que la forma de diseñar, implementar, probar y entregar software deje de depender de quién tome el ticket, y pase a ser un proceso asistido por inteligencia artificial, con evidencia citada en cada paso y con compuertas que **impiden** avanzar cuando falta algo.

Qué queremos lograr	Que la fase de desarrollo de una task — análisis, diseño, implementación, pruebas y apertura del PR — ocurra asistida por IA, y que el PR llegue a revisión acertado a la primera en todo lo que es verificable mecánicamente: estándar de código, cobertura de pruebas y completitud contra la task.
Dónde empieza y dónde termina	Empieza cuando el equipo implementador recibe una task de Jira. Termina con el PR abierto en GitHub, con la evidencia de lo desarrollado, la de las pruebas que lo cubren, y el match entre lo solicitado en la task y lo entregado, publicado como comentario del PR. La revisión del PR, el merge y el despliegue quedan fuera.
Por qué	Hoy, cuando un desarrollador declara terminado un desarrollo, empieza un ciclo de devoluciones en la revisión del PR. El estándar se aplica a mano, PR por PR; el conocimiento del sistema vive en pocas cabezas; y el costo del error se paga tarde.
Cómo	Nueve fases en orden de dependencia: primero el conocimiento , luego el estándar , luego el proceso , luego la automatización . Sobre esa base se cierran diseño, pruebas y entrega.
Bajo qué supuestos	El plan asume pruebas unitarias como alcance de automatización, y supone que los accesos, permisos y decisiones del cliente llegan en el momento en que se los necesita. El alcance de pruebas queda abierto a modificación : si el proyecto requiere integración, end-to-end u otros tipos, se incorporan revisando antes su impacto en el tiempo. Los supuestos y ese impacto están en la sección 7.
Decisión pedida	Aprobar el alcance y el orden de las fases, designar un dueño del proceso , y aceptar los cambios en la dinámica de trabajo de la sección 11.

2. El problema que resolvemos

Figura 1. El ciclo de devoluciones



El error no se origina al codificar: se origina antes, y solo se hace visible cuando alguien lo devuelve.

1. "Terminar el desarrollo" no es lo mismo que estar terminado

El desarrollador declara terminado, y ahí empieza la verdadera cola: el revisor devuelve el PR con observaciones, una y otra vez. Ese hueco entre **"ya terminé"** y **"ya pasó"** es donde vive el costo, y es el dolor que esta propuesta ataca de frente. La task envejece, el desarrollador cambia de contexto y vuelve, y el revisor gasta su tiempo — el más escaso del equipo — en observaciones que se repiten.

2. El alcance se descubre codificando

Las preguntas que debieron hacerse antes se terminan haciendo después, cuando cambiar de rumbo ya cuesta mucho más. La ambigüedad no desaparece por avanzar: se paga con intereses.

3. El conocimiento del sistema no está escrito

Depende de personas. El onboarding es lento, el impacto de un cambio se estima de memoria y la salida de una persona clave se siente como una pérdida de capacidad, no como un relevo.

4. El estándar técnico se aplica en el review, a mano

Las mismas observaciones se repiten PR tras PR porque nadie las verifica antes. El tiempo del revisor, que es el recurso más escaso, se consume en lo que una máquina podría validar.

5. La devolución es la factura de todo lo anterior

Hay dos devoluciones, y cuestan distinto. La primera y más frecuente son las **observaciones del revisor en el PR**: barata en comparación, pero repetitiva, y consume el tiempo del revisor en cosas que una máquina podría haber validado antes. La segunda es la **task devuelta por el equipo que testea**: ahí se paga de golpe todo lo que no se resolvió antes, porque hay que reabrir el contexto, reanalizar, reimplementar, volver a revisar y volver a desplegar. Juntas son la métrica más honesta del costo de la ambigüedad, y son las que esta guía busca mover.

Principio rector: el costo de corregir un error se multiplica aproximadamente diez veces por cada fase que avanza. Todo el diseño de esta guía existe para mover el hallazgo a las fases tempranas, donde corregir cuesta una conversación.

3. Qué queremos lograr

Cuatro capacidades encadenadas, sostenidas por una sola guía. No se trata de acelerar la escritura de código, sino de que la fase de desarrollo completa — desde entender la task hasta dejar el PR listo para revisión — ocurra con la misma disciplina cada vez.

El alcance, delimitado

Hoy intervienen **cuatro equipos** en la vida de una task, y el alcance de esta guía es exactamente el del segundo:

1. El equipo que **diseña la task** — define qué hay que hacer. **Fuera de alcance.**
2. El equipo que **implementa el código** para resolverla. **Aquí opera la guía.**
3. El equipo que **revisa el PR** — devuelve observaciones como comentarios del PR y, cuando queda conforme, lo integra y lo despliega a otros ambientes. **Fuera de alcance.**
4. El equipo que **testea** en esos ambientes y decide: aprueba para producción, o **devuelve la task** al equipo de desarrollo. **Fuera de alcance.**

Tres puertas de entrada, un solo proceso. El alcance es una **pasada**, y se entra por tres caminos:

- Una **task nueva** que entrega el equipo que la diseña.
- Las **observaciones del revisor** como comentarios en el PR. Es la devolución más frecuente y la primera que ocurre.
- Una **task devuelta** por el equipo que testea. Es la más costosa, porque ya pasó por todo.

La guía trata las tres igual: vuelven a pasar por el análisis y por la compuerta, con el contexto de la pasada anterior ya versionado. Reducir la segunda y la tercera puerta es, en la práctica, la forma más directa de medir si esta guía sirve.

Dentro. Todo lo que ocurre entre la recepción de la task por parte del equipo implementador y el PR abierto en GitHub: entender la task, contrastarla con el código, resolver los bloqueadores, diseñar la solución, planearla, implementarla, probarla, verificar que lo entregado corresponde a lo solicitado, y publicar el PR con esa evidencia.

El entregable final es el PR, y debe sostenerse por sí solo: el commit con la solución, la evidencia de las pruebas que la cubren, y un comentario que enfrenta uno a uno lo solicitado en la task contra lo entregado, incluyendo lo que quedó fuera y por qué.

Los límites son entregas entre equipos. A la entrada, la task diseñada que llega al desarrollador asignado. A la salida, el PR abierto que pasa al equipo revisor. La guía no reemplaza ni automatiza lo que hacen los otros tres equipos: se hace responsable del tramo del medio y les entrega evidencia para que su trabajo sea más corto y sus devoluciones más raras.

Figura 2. Las cuatro capacidades



En el día a día esto se traduce en algo simple: el desarrollador abre una sesión y ejecuta un solo comando. A partir de ahí, la guía conduce el ciclo y le pide a la persona exactamente lo que necesita de ella, ni más ni menos.

El objetivo concreto: acertar a la primera

Si la propuesta se resume en una sola cosa medible, es esta: **que el PR llegue a revisión sin las devoluciones que hoy se repiten**. No es una aspiración genérica. Cada tipo de observación que hoy escribe un revisor a mano tiene un mecanismo que la previene antes de que el PR exista.

Lo que hoy encuentra el revisor a mano	Qué lo previene antes de abrir el PR
Incumplimientos del estándar de código	Las reglas normativas se aplican durante la implementación, y la auto-revisión las audita una a una antes de publicar
Faltan pruebas, o la cobertura es insuficiente	La compuerta exige suite verde y cobertura sobre lo modificado. Sin eso no hay push
Quedó algo del alcance sin hacer	La verificación de match enfrenta uno a uno lo solicitado en la task contra lo entregado, y lo publica como comentario del PR
El PR es difícil de revisar	Un solo commit limpio con la solución completa, y la evidencia adjunta
Se resolvió sobre un supuesto equivocado	La compuerta de bloqueadores lo devuelve como pregunta antes de escribir código, no después

Hasta dónde llega la promesa. La guía puede acertar a la primera en todo lo **verificable mecánicamente**: el estándar, la cobertura, la completitud contra la task, la forma del PR. Ahí está la mayor parte del volumen de devoluciones, y es justamente lo que hoy se repite PR tras PR.

Lo que no puede garantizar es el juicio: si el enfoque elegido es el mejor para el negocio, o si el revisor prefiere otra solución igualmente válida. Esas devoluciones van a seguir existiendo, y está bien que existan — son la conversación técnica que sí aporta. **El objetivo no es un PR sin comentarios: es que los comentarios que queden sean los que valen la pena.**

Tres reglas que hacen esto confiable

Evidencia primero. Toda afirmación va citada contra el código. Lo que no se sabe se escribe como desconocido, nunca se inventa. Una ambigüedad es una pregunta bloqueante, no una decisión que el sistema tome por su cuenta.

Las compuertas son mecánicas. No dependen de la buena voluntad ni de la presión de entrega: si falta un artefacto, la escritura de código o la publicación quedan denegadas.

Lo humano se marca como humano. La guía nunca declara hecho un paso que le corresponde a una persona. Lo prepara, lo entrega y lo dice explícitamente.

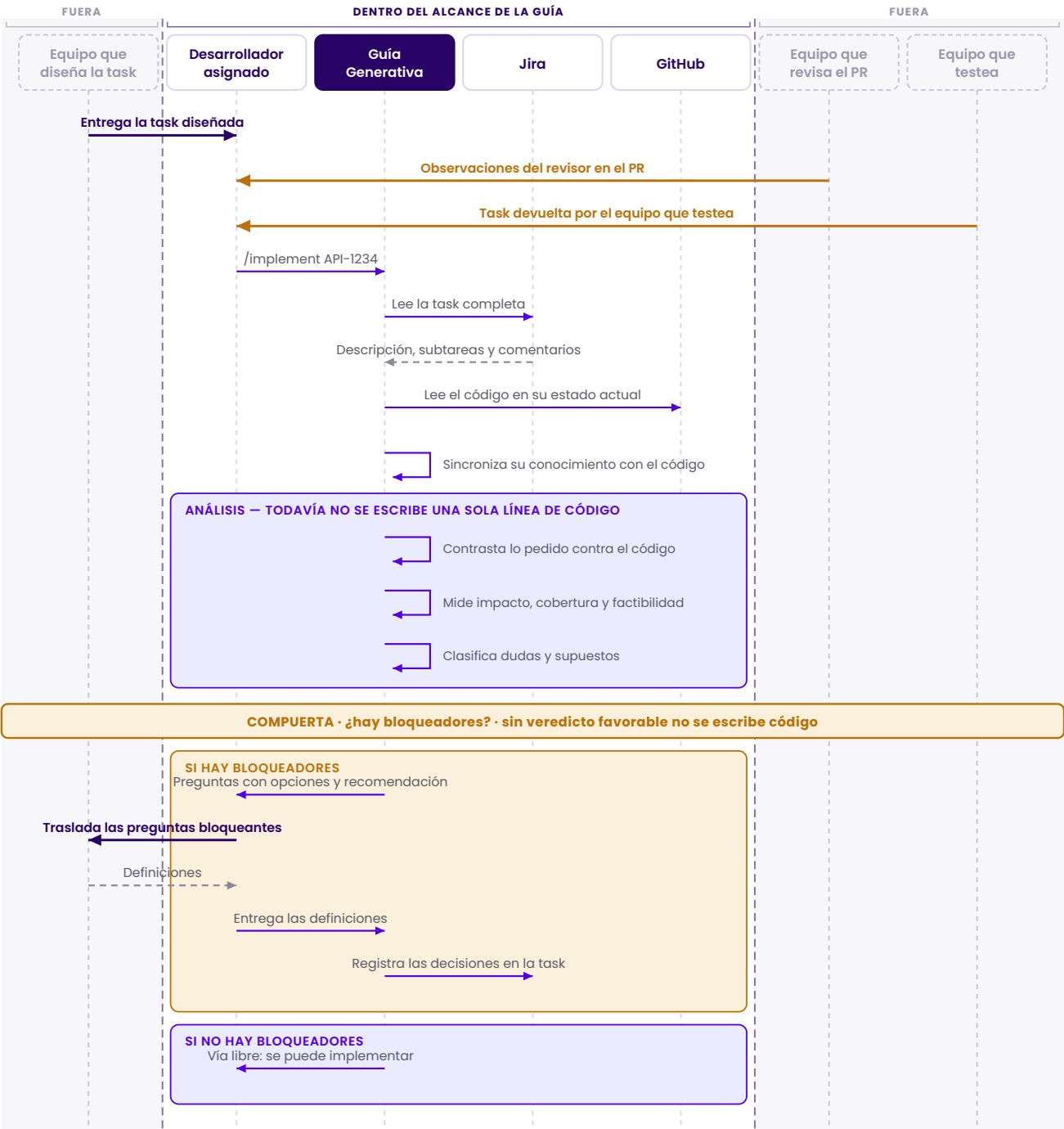
4. El flujo de trabajo completo, paso a paso

Las dos figuras siguientes muestran la secuencia real, con un carril por cada participante. Las **columnas grises de los extremos** son los otros tres equipos: quedan fuera del alcance y aparecen solo en las entregas, para que se vea con qué llegan y con qué se van. Todo lo que pasa entre las dos líneas punteadas es lo que esta guía se hace cargo de ejecutar.

Conviene leerlas atendiendo a **quién** actúa en cada momento: la persona decide, la guía ejecuta y deja evidencia. Las **compuertas** son las bandas ámbar que cruzan el diagrama: puntos de control donde el proceso se detiene si no se cumple una condición, y que no dependen de la disciplina de nadie.

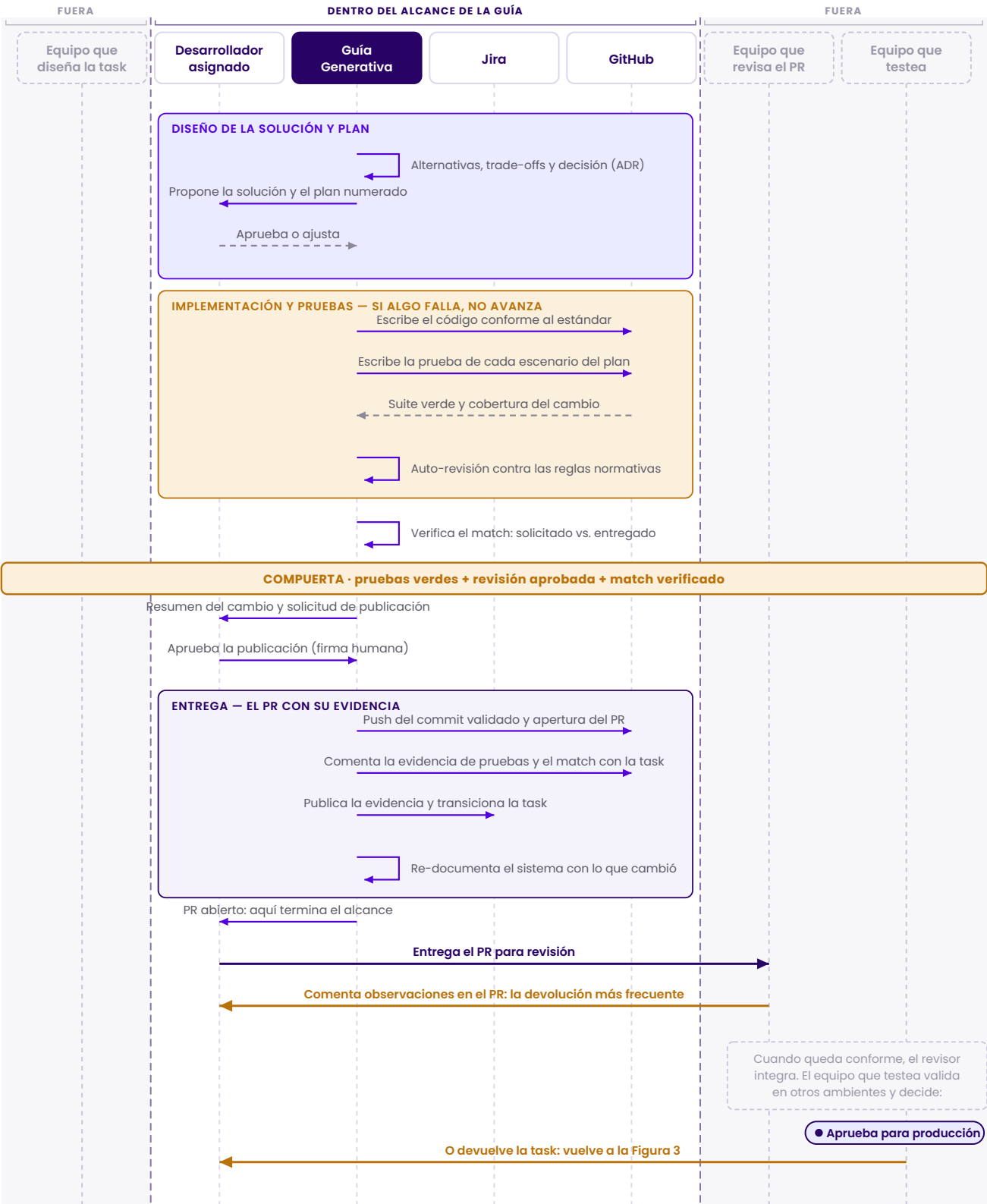
Las **flechas ámbar gruesas** son reproceso: trabajo que vuelve. Son dos caminos distintos — las observaciones del revisor en el PR y la task devuelta por el equipo que testea — y cada uno aparece en las dos figuras: en la 4 cuando sale, en la 3 cuando vuelve a entrar.

Figura 3. Las tres puertas de entrada, análisis y compuerta de bloqueadores



Se entra por una task nueva, por las observaciones del revisor o por una task devuelta, y el proceso es el mismo en los tres casos. Mientras existan bloqueadores, no se escribe una sola línea de código.

Figura 4. Diseño, implementación, pruebas y entrega del PR



Nada se publica sin pruebas verdes, revisión aprobada, match verificado y la firma explícita del desarrollador. El último acto de la guía es abrir el PR con su evidencia; de ahí en adelante deciden los otros equipos, y de sus decisiones salen las dos devoluciones.

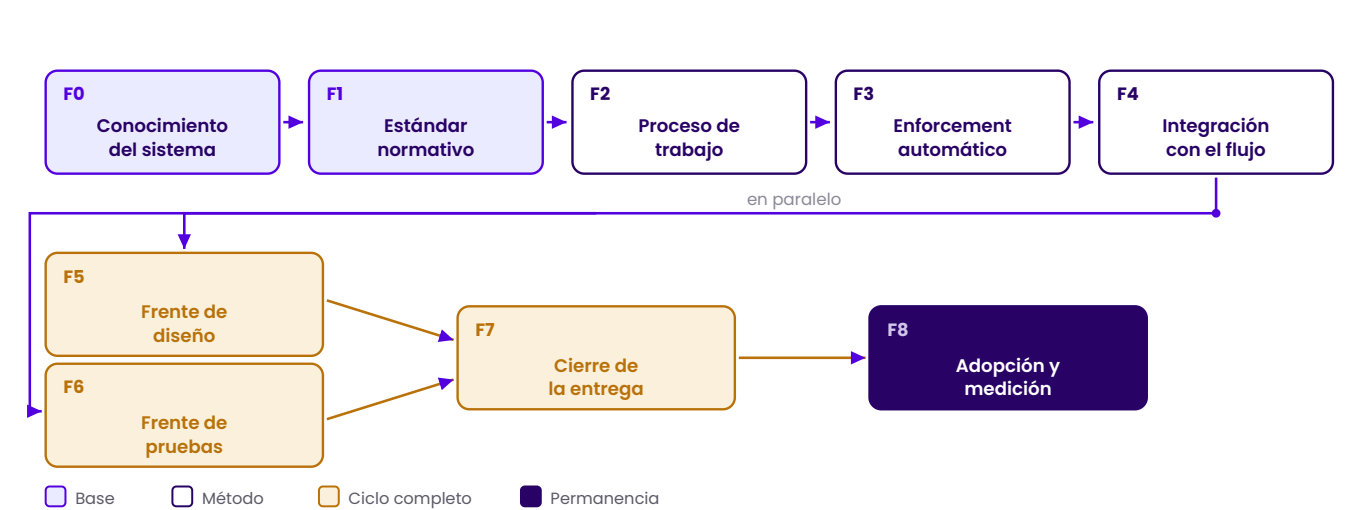
Lo que este flujo garantiza

Momento	Garantía
Al reentrar	Ni las observaciones del revisor ni una task devuelta se atienden de memoria: vuelven a pasar por el análisis y por la compuerta, con el plan y el contexto de la pasada anterior ya versionados.
Antes de codificar	Nadie escribe código sobre supuestos. La ambigüedad se devuelve como pregunta con opciones y recomendación; no se resuelve sola.
Durante	El estándar se cumple por construcción. El plan y las pruebas están numerados y trazados entre sí, uno a uno.
Antes de publicar	Suite verde, revisión aprobada y firma humana explícita. Ninguna de las tres es opcional ni se puede omitir por prisa.
Al entregar	El PR llega con la evidencia de las pruebas y con el match uno a uno contra la task. Quien revisa no tiene que reconstruir el contexto ni creer en la palabra de nadie.
Al cerrar	El conocimiento se actualiza solo, de modo que la siguiente task parte de un mapa vigente y no de uno degradado.
En cualquier punto	El trabajo está versionado: se retoma desde cualquier máquina y cualquier sesión, sin perder el contexto acumulado.

5. Las fases, y por qué en ese orden

El orden no es una preferencia de método: **cada fase es insumo de la siguiente**. Construir la automatización antes que el proceso, o el proceso antes que el conocimiento, produce trabajo que después hay que rehacer. Las fases F0 a F4 forman el camino crítico y son estrictamente secuenciales; a partir de F4 aparece el primer espacio real de paralelización.

Figura 5. Orden y dependencias de las nueve fases



F5 y F6 son las únicas fases que pueden ejecutarse en paralelo; ambas convergen en F7. F8 es la que sostiene todo lo anterior en el tiempo.

Bloque	Fases	Pregunta que responde
Base	F0, F1	¿Qué es este sistema y cómo se escribe código en él?
Método	F2, F3, F4	¿Cómo se trabaja, y cómo se garantiza que efectivamente se trabaje así?
Ciclo completo	F5, F6, F7	¿Cómo cerramos los frentes de diseño, pruebas y entrega?
Permanencia	F8	¿Cómo evitamos que todo esto se apague en seis semanas?

6. Detalle de cada fase y estimación

Base de la estimación: perfiles senior familiarizados con el dominio, a dedicación completa. Las cifras están en semanas-persona (sp) y son de orden de magnitud, pensadas para dimensionar la iniciativa; se afinan antes de comprometer fechas.

F0 · Base de conocimiento del sistema — 7 sp

Objetivo. Que el API deje de ser una caja negra: un mapa funcional y técnico donde cada afirmación está citada contra el código y lo desconocido está marcado como desconocido.

Entregables. Catálogo de conceptos de negocio; mapa de integraciones externas; arquitectura y patrones; mapa de dependencias entre módulos; catálogo de códigos de error; mecanismo de sincronización entre documentación y código.

Por qué va primero. Todo lo demás razona sobre esto. Un proceso excelente sobre conocimiento inexistente produce planes equivocados con mucha ceremonia. **Cómo se organiza:** el trabajo se corta por módulos, que es la unidad natural, y lo documentado se audita de forma cruzada. Esa auditoría es la que sostiene la disciplina de citar.

F1 · Estándar normativo de implementación — 3 sp

Objetivo. Que "cómo se escribe código aquí" sea un conjunto de reglas explícitas y numeradas, y no el criterio de quien revisa ese día.

Entregables. Reglas normativas de estilo, arquitectura, robustez y proceso, extraídas del histórico real de revisiones de PR; convenciones y patrones documentados.

Por qué va aquí. Es la vara con la que se implementa (F2) y con la que se revisa (F3). Definirla después obliga a rehacer planes y revisiones ya emitidos. El trabajo se corta por familias de reglas.

F2 · Proceso de trabajo — 4 sp

Objetivo. Que un ticket siempre recorra los mismos pasos, con criterio de salida explícito en cada uno y artefactos de nombre fijo.

Entregables. Las fases del ciclo con su criterio de salida; plantillas de cada artefacto; taxonomía de hallazgos y severidades; trazabilidad numerada entre plan y pruebas; escalamiento del rigor según riesgo (trivial, normal, riesgoso); rutas especiales para incidentes en producción.

Por qué va aquí. Es el diseño de lo que después se automatiza. Automatizar un proceso que no ha sido diseñado equivale a codificar el caos. **Cómo se organiza:** la redacción del proceso y la construcción de plantillas y taxonomías son trabajo separable, y avanzan a la vez.

F3 · Enforcement automático — 2 sp

Objetivo. Que las compuertas no dependan de la buena voluntad.

Entregables. Guardianes que deniegan escribir código sin el análisis aprobado, y que deniegan publicar sin pruebas verdes, revisión aprobada y aprobación explícita de una persona; anclaje de la aprobación al commit exacto que fue validado; protección de las firmas humanas; suite de pruebas del propio enforcement.

Por qué va aquí. Sin esto, bajo presión de entrega el proceso se salta. Con esto, sencillamente no se puede. Los guardianes y su suite de pruebas son trabajo separable.

F4 · Integración con el flujo real de trabajo — 2 sp

Objetivo. Cero fricción de arranque: un comando, y el resto ocurre.

Entregables. Puente con Jira (lectura automática del ticket, escritura siempre detrás de aprobación); preparación automática del espacio de trabajo por ticket; progreso versionado y reanudable desde cualquier máquina; índice de trabajos por clave de ticket; varios tickets en curso sin interferencia entre sí.

Por qué va aquí. Determina si el equipo lo usa o lo evade. Un proceso correcto con un arranque costoso no se adopta.

F5 · Frente de diseño — 3 sp

Objetivo. Que entre la task recibida y la primera línea de código exista una solución **diseñada**: alternativas evaluadas, trade-offs explícitos y decisión registrada. No cubre la definición del requerimiento — eso ocurre antes de que la task llegue.

Entregables. Fase de diseño dentro del proceso; plantilla de registro de decisión de arquitectura acotada al alcance de una task; catálogo de decisiones que se acumula task a task; criterio explícito de cuándo una task exige revisión de diseño con el tech lead.

Beneficio. Las decisiones dejan de repetirse y de discutirse dos veces. **Se ejecuta en paralelo con F6.**

F6 · Frente de pruebas — 3 sp

Objetivo. Que la evidencia de calidad sea un resultado verificable por máquina, no un texto que alguien redactó.

Entregables. Generación asistida de pruebas a partir de la matriz de escenarios del plan; medición de cobertura sobre lo efectivamente modificado; integración con QA; criterios de aceptación funcional trazados al ticket.

Alcance de esta estimación: pruebas unitarias. Es el punto de partida, porque son las que no exigen infraestructura adicional. Incorporar integración o end-to-end cambia el esfuerzo de esta fase — el detalle está en la sección 7.

Beneficio. Menos defectos escapados, y la prueba deja de ser lo primero que se sacrifica cuando aprieta la fecha.

F7 · Cierre de la entrega: el PR como evidencia — 2 sp

Objetivo. Que el PR sea autosuficiente. Quien lo revisa debe poder juzgarlo sin reconstruir el contexto: qué se pidió, qué se hizo, con qué pruebas quedó cubierto y en qué se diferencia una cosa de la otra.

Entregables. PR con un solo commit limpio que contiene la solución completa; evidencia de las pruebas ejecutadas y de la cobertura sobre lo modificado, adjunta al PR; **comentario en el PR que enfrenta uno a uno lo solicitado en la task contra lo entregado**, con lo que quedó fuera y por qué; publicación de esa evidencia en la task de Jira y su transición de estado; re-documentación automática del sistema con lo que cambió.

Beneficio. El tiempo del revisor se dedica a juzgar la solución, no a averiguar qué se hizo. Y el conocimiento construido en F0 se mantiene vivo solo, en lugar de degradarse mes a mes. **Fuera de esta fase:** el merge, el despliegue, los runbooks de release y la observabilidad en producción.

F8 · Adopción, formación y medición — 3 sp

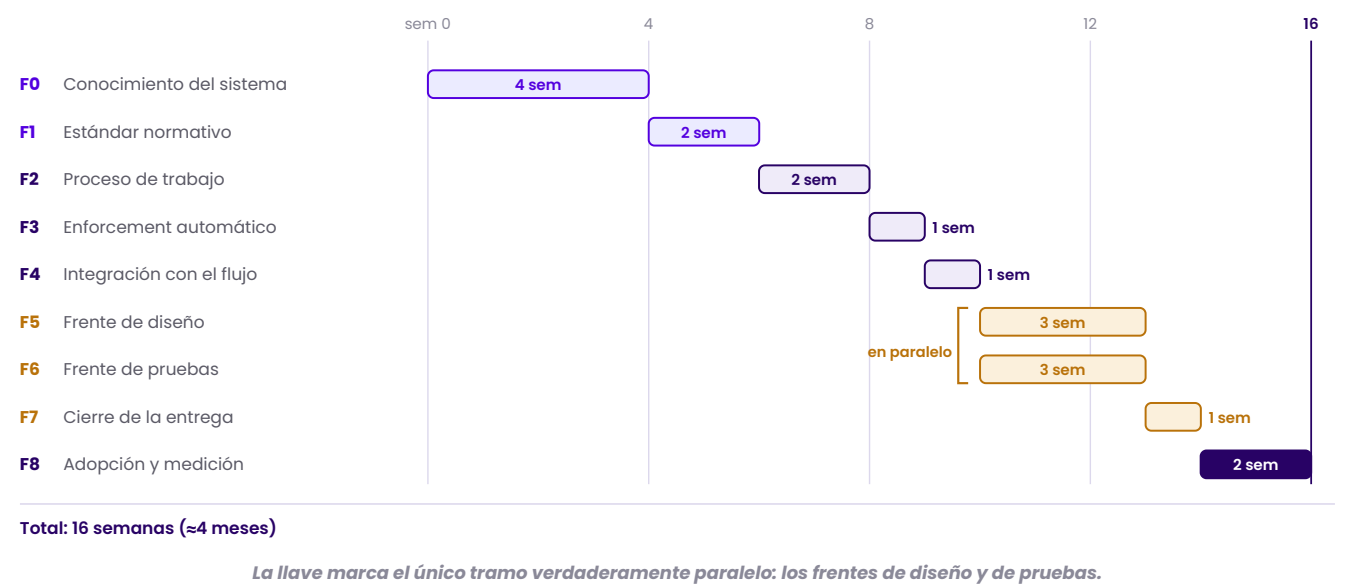
Objetivo. Pasar de "funciona con quien lo construyó" a "así trabaja el equipo".

Entregables. Secuencia de adopción por etapas (plantilla de PR, definición de listo y de terminado, matriz de impacto obligatoria, revisión de diseño, listas de verificación); formación del equipo; tablero de indicadores; retrospectiva mensual del proceso; el **set de evaluación** con el que después se detecta la degradación (sección 9); y el dueño del proceso formalmente designado.

Por qué va al final y por qué no es opcional. Sin dueño y sin medición, cualquier proceso se apaga en semanas. Esta fase es la que protege toda la inversión anterior, y continúa como esfuerzo recurrente después de la semana 16.

7. Cronograma y esfuerzo

Figura 6. Cronograma del plan de trabajo



Fase	Esfuerzo (sp)	Calendario	Semanas
F0 · Conocimiento del sistema	7	4 sem	0 – 4
F1 · Estándar normativo	3	2 sem	4 – 6
F2 · Proceso de trabajo	4	2 sem	6 – 8
F3 · Enforcement automático	2	1 sem	8 – 9
F4 · Integración con el flujo	2	1 sem	9 – 10
F5 · Frente de diseño	3	3 sem	10 – 13
F6 · Frente de pruebas	3	en paralelo	10 – 13
F7 · Cierre de la entrega	2	1 sem	13 – 14
F8 · Adopción y medición	3	2 sem	14 – 16
Total	29	16 sem	≈4 meses

El plan reserva cerca del 10% de holgura entre la capacidad disponible y el trabajo estimado. No es margen sobrante: absorbe coordinación, auditoría cruzada e interrupciones. Un plan al 100% de utilización no es un plan, es un deseo.

El alcance de automatización de pruebas mueve la estimación

Conviene decirlo antes de que aparezca como sorpresa: **esta estimación toma las pruebas unitarias como punto de partida**, y nada más. El esfuerzo de F6 — y con él el total — depende de qué niveles de prueba se decida automatizar, y esa decisión todavía no está tomada.

Nivel de prueba	Qué arrastra construirlo	En esta estimación
Unitarias	La suite dentro del código fuente y su ejecución determinista en local y en CI	Incluido
Integración	Datos de prueba, dobles de los servicios externos, un ambiente reproducible	No incluido
End-to-end	Ambiente completo, datos estables, orquestación y tiempos de ejecución largos	No incluido
Contrato, carga, seguridad	Herramental propio y criterios de aceptación específicos por cada tipo	No incluido

El plan está abierto a incorporarlos. Ninguno de estos niveles queda excluido por diseño: si el proyecto los requiere, se suman al alcance revisando antes su impacto en el tiempo, de modo que la decisión se tome con el costo a la vista y no después. Lo que no cambia es el mecanismo — la matriz de escenarios del plan y la compuerta de suite verde funcionan igual en cualquier nivel; lo que crece es lo que hay que construir debajo. Por eso cada nivel adicional se dimensiona cuando se decida incluirlo: su costo lo determinan los ambientes y los datos disponibles del proyecto, no la guía.

Qué puede mover estas fechas

Las 16 semanas suponen que el equipo puede avanzar sin esperar. En la práctica, una parte del plan depende de cosas que el equipo no controla: permisos, accesos y decisiones que necesitan a otras personas en la mesa. Conviene ponerlas por delante, no descubrirlas en la semana seis.

Qué se necesita del cliente	Cuándo deja de ser espera y pasa a ser bloqueo	Efecto en el calendario
Permisos y accesos: repositorio, Jira, ambientes, histórico de PR	Semana cero, antes de F0	Cada semana de demora corre el plan completo una semana
Decisión sobre el alcance de automatización de pruebas	Antes de la semana 10, cuando arranca F6	Si se define después, F6 se replanifica; si suma niveles, la estimación se recalcula
Reuniones de decisión con participantes de varias áreas	F2, F5 y F8	Agendar tres áreas suele costar más que ejecutar la decisión misma
Respuestas a preguntas bloqueantes durante el piloto	En cualquier momento	La compuerta detiene la task: el reloj corre sin que el trabajo avance
Designación del dueño del proceso	Antes de la semana 14, cuando arranca F8	Sin dueño, F8 no cierra y la inversión anterior queda sin sostener
Aprobación de licencias y cupo de uso de IA	Antes de F8	Limita el escalamiento al equipo completo
Adopción de las condiciones C-1 a C-6 (sección 11)	C-1 antes de F6; el resto durante F8	Sin C-1 no hay evidencia que automatizar y F6 queda sin sustento

Condición de validez de esta estimación. Las 16 semanas son válidas en la medida en que el cliente pueda sostener la disponibilidad que el plan supone: accesos otorgados en la semana cero, decisiones tomadas en la ventana en que se necesitan, personas disponibles para las reuniones de definición, y las condiciones de la sección 11 adoptadas en su momento. No es una salvedad de trámite: la mayor parte del riesgo de calendario de esta iniciativa no está en la construcción, está en la espera. Cuando una de estas dependencias se corre, el calendario se recalcula en ese momento — no cuando el atraso ya se acumuló.

8. Cómo mediremos que funcionó

Indicador	Meta	Cómo se mide
Ciclos de revisión por PR	-50%	Idas y vueltas entre el PR abierto y su aprobación. La meta operativa es que el primer ciclo alcance
Tasks devueltas por el equipo que testea	-50%	Devoluciones sobre tasks entregadas, por período
Retrabajo por alcance mal entendido	-50%	Tasks reabiertas o cambios de alcance posteriores al plan
Defectos escapados a producción	-40%	Incidentes con causa evitable
Tiempo de ciclo por ticket	-30%	Jira: de "en progreso" a "terminado"
Tiempo hasta el primer review	-50%	PR abierto hasta primer comentario
Observaciones repetidas en PR	-70%	Hallazgos bloqueantes por PR
Onboarding hasta productividad	Meses a semanas	Primer PR integrado de un nuevo integrante

Los dos primeros indicadores son los que importan. Miden las dos devoluciones, que son la medida más directa y menos discutible de si esta guía funciona: son números que ya existen en GitHub y en Jira, que nadie tiene que inventar, y que caen solo si el problema se resolvió antes. El resto explica **por qué** se mueven. Conviene además separar, al mirar la tendencia, las observaciones de forma de las de criterio técnico: la guía debe llevar las primeras casi a cero, y no pretende eliminar las segundas.

La línea base se levanta antes de F0. Sin ella no hay forma de demostrar el retorno, y la conversación se vuelve una discusión de percepciones. La regla es medir el **efecto**, no el cumplimiento: si el proceso no mueve estos números, se ajusta o se poda. Hay además beneficios que no se expresan en porcentaje y son igual de reales: el conocimiento deja de ser tácito, cada línea de código se remonta a un plan, un análisis y un ticket, y se reduce de forma sostenida la dependencia de personas clave.

9. Capacidad designada para soporte y evolución

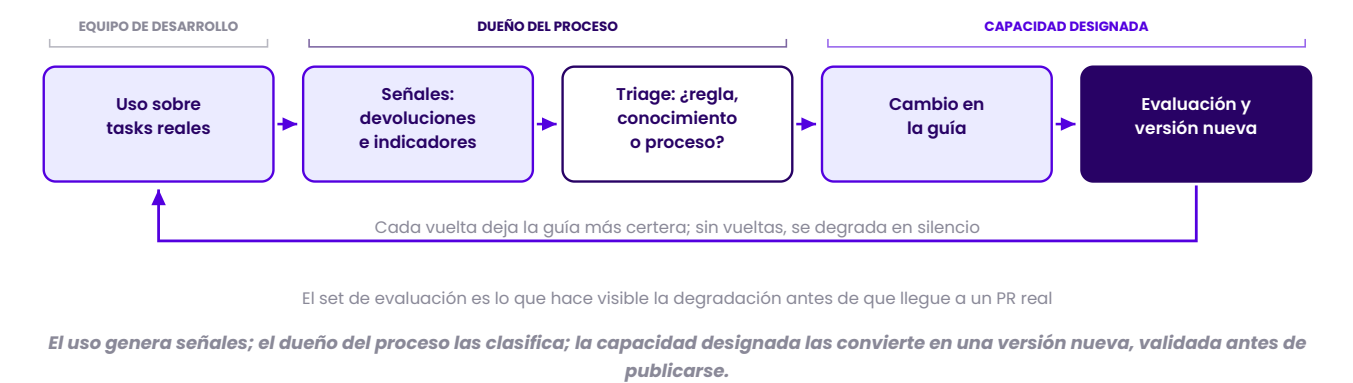
Una guía generativa no es un entregable que se congela. El conocimiento del sistema envejece, el estándar se queda corto, el proceso acumula deuda y el modelo cambia de comportamiento. Y a diferencia del software convencional, esto **no falla con un error visible: se degrada en silencio**, y se nota cuando un PR malo llega a revisión. Por eso, a partir de la semana 16 el plan no propone soporte abierto, sino una **capacidad designada**: una asignación dimensionada y acotada que cubre, en un mismo cupo, el soporte de lo entregado y la evolución de la guía.

Por qué un cupo único y no dos bolsas separadas. Separar "garantía" de "mejoras" obliga a discutir, en cada pedido, de cuál de las dos sale. Esa discusión desgasta la relación y no le aporta nada al producto. Con un cupo único no hay clasificación que negociar: el dueño del proceso prioriza, la capacidad se consume y el consumo queda a la vista.

Qué cubre

Naturaleza	Ejemplos
Soporte — que lo entregado siga funcionando	Defectos en el proceso o en las compuertas; falsos positivos del enforcement que frenan trabajo legítimo; ajustes cuando el repositorio, Jira o el pipeline cambian de forma
Evolución — que la guía siga al día	Reglas normativas nuevas surgidas de la revisión; documentar módulos que quedaron fuera del alcance priorizado de F0; poda del proceso que resultó ruido; cambio de versión del modelo y su re-evaluación; capacidades nuevas

Figura 7. El ciclo de evolución



El motor: cada devolución que la guía debió prevenir es un defecto de la guía

No del desarrollador. Ese cambio de atribución es lo que convierte el indicador de la sección 8 en un motor de mejora con destino concreto:

- Devolución **de forma** que la guía no previno → falta una regla normativa, hay un hueco de conocimiento, o falta una compuerta. Entra al backlog de la guía con un destino identificado.
- Devolución **de criterio** → no es defecto de la guía. Si sienta precedente, se registra como decisión de arquitectura y pasa a formar parte del catálogo.

El ritmo que sostiene

Cadencia	Qué se hace	Quién
En cada task	Re-documentación automática del sistema con lo que cambió	La guía (F7)
Semanal	Triage de lo que la guía debió prevenir y no previno	Dueño del proceso
Mensual	Retrospectiva con los indicadores de la sección 8; poda de lo que resultó ruido	Dueño del proceso y capacidad designada
Trimestral	Revisión del estándar y de la versión del modelo; corrida del set de evaluación	Capacidad designada

Cómo se detecta la degradación: el set de evaluación

Un conjunto acotado de tasks ya resueltas que se vuelve a correr cada vez que cambia el modelo, el proceso o las reglas. Si el análisis deja de detectar bloqueadores que antes detectaba, o el plan deja de cubrir escenarios que antes cubría, **se ve ahí y no en un PR real**.

Es la única forma de que la degradación silenciosa deje de ser silenciosa, y es la razón por la que el set se construye durante el proyecto y no después: sin él, el primer cambio de modelo no tiene contra qué compararse.

Qué acota esta capacidad

- Se dimensiona en **días-persona por mes** y se acuerda por un **plazo definido, renovable**. El piso lo fija el propio ritmo: el triage semanal, la retrospectiva mensual y la revisión trimestral consumen capacidad aunque no haya ninguna mejora pendiente.
- Lo que no entra en el cupo del mes se prioriza para el siguiente, o se amplía el cupo de forma explícita. La decisión es siempre del dueño del proceso, con el consumo del período a la vista.
- No está incluida en el esfuerzo ni en el alcance de las 16 semanas.** La semana 16 entrega la versión 1 de la guía, operando sobre tasks reales. Esta capacidad es un acuerdo aparte, y su dimensionamiento se define con el volumen real de tasks del proyecto.

10. De qué depende

Dependencia	Quién	Criticidad	Si no se cumple
Equipo a dedicación completa durante las 16 semanas	Dirección	Alta	El calendario se estira de forma proporcional a la dedicación real
Acceso al repositorio de código y a Jira	Cliente / DevOps	Alta	El ciclo opera en modo manual, con fricción alta
Un dueño del proceso designado, más allá de la semana 16	Dirección	Alta	La guía se degrada en semanas (F8)
Disponibilidad de PO y tech lead para responder preguntas bloqueantes	Negocio	Alta	Las compuertas detienen tickets. Es el comportamiento correcto, pero frena el flujo
Acceso al histórico de revisiones de PR	Cliente	Media	FI se construye por criterio y no por evidencia
Suite de pruebas ejecutable en local y en integración continua	Equipo	Alta	Bloquea F6 por completo — es la condición C-1 de la sección 11
Tickets con una definición de listo mínima	Equipo	Media	Más ciclos en la etapa de análisis
Licencias y cupo de uso de IA para el equipo	Dirección	Media	Limita el escalamiento en F8
Ventana de formación, aproximadamente cuatro horas por persona	Equipo	Baja	Adopción más lenta

Cada una de estas dependencias tiene un momento a partir del cual deja de ser una espera y pasa a ser un bloqueo con costo en calendario. Ese detalle, con su efecto concreto sobre las fechas, está en la sección 7.

11. Cambios que requiere la dinámica de trabajo

La guía no funciona sobre cualquier forma de trabajar. Hay prácticas del proyecto que **deben cambiar** para que pueda garantizar lo que promete, y no son preferencias metodológicas: son condiciones técnicas de las que depende que la evidencia sea verificable por máquina y no la afirmación de alguien.

Cada una queda marcada con un identificador, para poder seguirla como tema aparte con su propio responsable y su propia fecha.

C-1 · Las pruebas automatizadas deben ser obligatorias dentro del código fuente

Por qué la guía lo necesita: la compuerta de publicación exige suite verde como evidencia verificable. Si las pruebas no viven en el repositorio y no se ejecutan con un comando, no hay nada que la guía pueda verificar ni adjuntar al PR. **Si no cambia:** la evidencia de calidad vuelve a ser una afirmación humana, la compuerta más importante deja de existir y el PR autosuficiente se cae. **Es la única de estas condiciones que debe estar resuelta antes de que F6 arranque.**

C-2 · La task debe cumplir una definición de listo mínima

Por qué: el análisis contrasta lo pedido contra el código. Sin un enunciado analizable, la compuerta bloquea y la task rebota antes de empezar. **Si no cambia:** más ciclos de preguntas al inicio, y el tiempo de ciclo empeora antes de mejorar.

C-3 · Las decisiones se escriben en la task, no solo se conversan

Por qué: la task es la memoria del proyecto. La guía la lee al arrancar, y una task devuelta se retoma desde ahí. **Si no cambia:** cada reproceso vuelve a discutir lo que ya se había decidido.

C-4 · El PR lleva un solo commit limpio y la evidencia como comentario

Por qué: es el formato del entregable, y es lo que ancla la aprobación al commit exacto que fue validado. **Si no cambia:** hay que rediseñar el cierre de la entrega y se pierde la trazabilidad entre lo aprobado y lo publicado.

C-5 · Las preguntas bloqueantes tienen responsable y plazo acordado

Por qué: la compuerta detiene la task hasta que se responden. Es el comportamiento correcto, pero sin un acuerdo de respuesta se vuelve un cuello de botella. **Si no cambia:** tasks bloqueadas por tiempo indefinido, y el equipo aprende a rodear el proceso.

C-6 · El equipo que testea registra la causa de cada devolución

Por qué: la tasa de devolución es uno de los dos indicadores principales de la sección 8. Sin causa registrada, el número sube o baja sin que se pueda atribuir a nada. **Si no cambia:** no hay forma de demostrar el retorno con datos propios del proyecto.

Estas condiciones no se imponen el primer día: F8 las introduce por etapas y cada una necesita un dueño identificable. La excepción es C-1, que es una precondition y no una meta de adopción: sin pruebas automatizadas en el código fuente no hay nada que automatizar, y F6 no tiene sobre qué operar.

12. Riesgos y mitigación

Riesgo	Mitigación
Los permisos y accesos llegan tarde	Se piden y se verifican en la semana cero, antes de comprometer fechas. El plan arranca por lo que no requiere acceso, y cada semana de demora se traslada explícitamente al calendario en vez de absorberse en silencio.
Las decisiones que involucran a varias áreas se demoran	Cada decisión pendiente lleva fecha límite y una opción por defecto documentada , que se aplica si no hay respuesta. Toda decisión tomada por defecto queda registrada como supuesto, visible y reversible.
El alcance de automatización de pruebas se decide tarde, o crece	Se decide antes de la semana 10. Si crece por encima de las unitarias, se recalcula y se cotiza antes de continuar, no después. La sección 7 deja escrito qué arrastra cada nivel.
La disponibilidad para reuniones de decisión es menor que la supuesta	Las decisiones se preparan por escrito con opciones y recomendación, para que la reunión sirva para decidir y no para explicar. Lo que se puede resolver de forma asíncrona no se agenda.
Las condiciones C-1 a C-6 no se adoptan	Cada una tiene responsable y fecha desde el arranque. C-1 es precondition de F6: si no está lista, F6 no arranca y se replanifica antes de gastar el esfuerzo.
F0 se desborda porque el sistema es más grande de lo estimado	Se prioriza por los módulos de mayor tránsito y el resto se documenta bajo demanda. Todo lo no cubierto queda marcado de forma explícita como desconocido, de modo que la guía lo trate como pregunta bloqueante y no como dato. El hueco se ve; no se disimula.
La guía se degrada después de la entrega	La capacidad designada de la sección 9 existe para eso, con un ritmo fijo y un set de evaluación que hace visible la degradación antes de que llegue a un PR.
La salida de un integrante detiene el proyecto	Auditoría cruzada desde F0: cada frente es revisado por alguien distinto de quien lo produjo, así que nadie es el único que entiende su parte.
Se percibe como burocracia y el equipo lo rechaza	El rigor escala al riesgo: un cambio trivial no lleva registro de decisión ni revisión de diseño. La adopción es por etapas (F8), no "todo el lunes".
El conocimiento queda obsoleto y la IA razona sobre datos viejos	La sincronización es obligatoria y verificada; F7 hace que cada cierre re-documente el sistema.
La guía afirma lo que no sabe	Regla no negociable de evidencia: todo citado contra el código, lo desconocido marcado como tal, y la ambigüedad convertida en pregunta bloqueante.
Se salta una compuerta por presión de entrega	Las compuertas deniegan la operación (F3). No dependen de disciplina individual.
El enforcement genera falsos positivos y frena el trabajo	Calibración con tickets reales antes de escalar al equipo completo.
La formación de F8 no alcanza para la adopción	El acompañamiento se hace sobre trabajo real en lugar de en aula, y la retrospectiva mensual detecta temprano si el equipo no está usando la guía.

13. Próximos pasos

1. **Validar el alcance** — en particular sus dos límites: la task recibida y el PR abierto — y el orden de las nueve fases.
2. **Designar el dueño del proceso.** Sin dueño, no arranca.

3. **Levantar la línea base** de los siete indicadores de la sección 8.
4. **Decidir el alcance de automatización de pruebas** (sección 7). Si pasa de unitarias, la estimación se recalcula antes de comprometer fechas.
5. **Asignar responsable y fecha a cada condición C-1 a C-6** (sección 11), empezando por C-1.
6. **Fijar la semana cero**, tramitar permisos y accesos, y acordar el foro y la cadencia donde se toman las decisiones que involucran a varias áreas.
7. **Dimensionar la capacidad designada** para soporte y evolución (sección 9), y su plazo inicial.
8. **Arrancar F0.**

Documento vivo. El alcance de las fases, los indicadores y las estimaciones se afinan con el equipo antes de comprometer fechas. Las cifras de esfuerzo dimensionan la iniciativa; el precio se cotiza una vez aprobado el alcance. Las 16 semanas son válidas bajo los supuestos declarados en la sección 7 — alcance de pruebas unitarias, y disponibilidad del cliente para accesos, decisiones y definiciones en la ventana en que el plan los necesita.

Documento confidencial. Elaborado por Tech and Solve para Viva Aerobus. Propuesta de guía generativa para el ciclo de desarrollo del proyecto Generic API, agosto de 2026.